Syfte

Syftet med projektet var att lära mig grunderna i API-hantering, HTTP-anrop och JSON-parsing i Python genom att bygga en applikation i en modulär struktur. Projektet demonstrerar hur man hämtar och kombinerar extern data från två olika REST API-endpoints (mapping och realtidspriser) för att söka fram ett specifikt föremål, beräkna High Alchemy-vinst och presentera resultatet med strukturerad loggning.

Området och dess relevans

API-hantering är väldigt viktigt inom Data Science-rollen då data i alla dess former är grunden i arbetet, oavsett användningsområde. I detta program tar jag fram priser på olika items i RuneScape. Det gör att man kan få aktuell information och prisdata i realtid, vilket man i sin tur kan använda för att göra olika analyser eller bygga modeller (exempelvis för att prediktera priser i framtiden). Att behärska API-anrop gör att man självständigt kan samla in data utan att vara beroende av färdiga dataset i CSV- eller Excel-format.

Viktiga begrepp

• REST **API:** Ett standardiserat gränssnitt för att överföra data mellan en klient (mitt program) och en server över HTTP.

• Endpoint**:** En specifik webbadress/sökväg i API:et där vissa data finns tillgänglig (t.ex. `/mapping` för item-data eller `/latest` för priser).

• JSON **(JavaScript Object Notation):** Det textbaserade dataformat som API:er vanligtvis returnerar data i, vilket enkelt kan omvandlas till `dicts` och `lists` i Python.

• **User-Agent:** En identifikationssträng i HTTP-headern som talar om för servern vem som gör anropet. Krävs av OSRS Wiki för att förhindra överbelastning.

• **Logging:** Ett sätt att i koden registrera händelser, fel och information under körning i stället för att använda vanliga `print`-satser.

Genomförande

Projektet byggdes i Python med en ren och modulär struktur uppdelad på tre filer under en src/-mapp samt en huvudfil:

- **src/models.py:** Innehåller en Item-dataklass som samlar objektets ID, namn, High Alch-värde samt köp- och säljpris från Grand Exchange. Klassen beräknar även alch-marginalen.
- **src/api_client.py:** Ansvarar för alla HTTP-anrop mot OSRS Wiki API ([https://prices.runescape.wiki/api/v1/osrs](https://prices.runescape.wiki/api/v

1/osrs)). Här sätts den krävda User-Agent-headern för att hantera godkända anrop mot endpoints som /mapping och /latest.

- **src/service.py:** Hanterar affärslogiken. Den matchar användarens söksträng mot alla föremålsnamn från mapping-anropet och kombinerar detta med färska prisdata från senast hämtade priser.
- **main.py:** Projektets startpunkt som tar emot användarinput via terminalen, konfigurerar loggern och skriver ut sökresultatet med hjölp av logging.

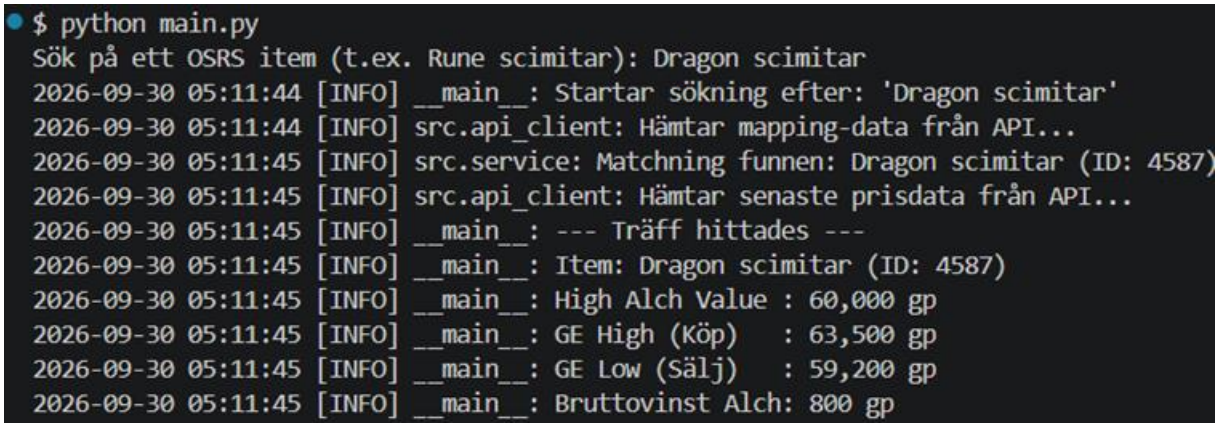
Problem och hantering:

Ett problem som uppstod var att OSRS Wiki API kräver en anpassad User-Agent, annars stoppas anropen med felkod 403 Forbidden. Detta löstes genom att lägga till projektidentifikation och kontaktuppgifter i HTTP-headers.

Använda bibliotek: requests för API-hantering och Pythons inbyggda logging-modul.

Resultat

Programmet tar emot en sökning från användaren (t.ex. *"Rune scimitar"*), gör anrop mot API:et i bakgrunden och presenterar köp- och säljpriser samt High Alch-värde direkt i terminalen via loggern:



Koppling till yrkesrollen

Nu är priser på RuneScape-items inte direkt avgörande för yrkesrollen i sig. Men principen — att automatiskt kunna ladda hem och aktivt uppdatera priser på relaterade tillgångar — är helt central. Byt ut priser på RuneScape-items mot aktiekurser, råvarupriser på olja eller valutakurser, så har du grunden till ett program som kan bevaka prisutveckling i realtid. Det är exakt detta en Data Scientist eller Data Analytiker gör när de bygger automatiska datapipelines för att samla in underlag till prediktionsmodeller och beslutsstöd.

Begränsningar och möjliga förbättringar

En av de tydligaste förbättringarna mitt projekt hade kunnat göra är att spara ner API-hämtningen av sidans information i en lokal databas eller cache-fil. Det skulle göra att man mycket snabbare och mer dataeffektivt kan leta upp andra items utan att behöva ladda ner hela mapping-listan på nytt vid varje sökning. Jag valde aktivt att inte använda mig utav en databas för att hålla mitt scope fokuserat på rena API-förfrågningar.

Hade jag velat utöka scopet på projektet hade jag även kunnat använda mig av endpoints som /1h och /5m för att få reda på prisförändring över tid, eller /timeseries för att hämta historisk data. Datan hade sedan kunnat läggas in i ett diagram (med t.ex. matplotlib) för att analysera trendbrott och ta beslut därifrån. En annan förbättring är att lägga till "fuzzy matching" så att programmet hittar rätt item även om man stavar fel eller söker på förkortningar.

Källor

- OSRS Wiki Real-time Prices API Documentation: https://oldschool.runescape.wiki/w/RuneScape:Real-time_Prices
- Python Requests Library: https://requests.readthedocs.io/
- Python Logging Facility: https://docs.python.org/3/library/logging.html